Java Competence: General Description

One description covering all levels. Levels are cumulative: a higher level includes everything from the lower levels.

1. Purpose and scope

This document describes what it means to be competent in Java programming. It is used as reference material for creating and reviewing test questions. It does not split the competence into separate documents per level. Instead, each topic below says what is expected at Level 1 and what is added at Level 2. Questions must stay within the topics and depth described here.

2. What Java competence means

A competent Java developer can write, read, test and maintain programs in Java (version 17 or newer), use the standard library and common tooling, and build simple backend applications with Spring Boot and a relational database. Competence is shown by correctly applying concepts to short, realistic tasks, not only by recalling definitions.

3. Competence areas by level

Area	Level 1: foundation	Level 2: adds
Language fundamentals	Primitive and reference types, variables, operators, conditionals, loops, methods and parameters, strings, arrays, basic input and output, `main`.	Type conversion and casting, `var`, switch expressions, text blocks, `String.format` and `formatted`, record classes, reading short unfamiliar code and predicting its output.
Object-oriented programming	Classes and objects, fields, constructors, encapsulation (private fields, getters and setters), inheritance with `extends`, `this` and `super`, method overriding and overloading.	Interfaces and `implements`, abstract classes, polymorphism, composition versus inheritance, static versus instance members, immutability, `equals` and `hashCode`, choosing a suitable design for a small problem.
Collections and generics	Choosing and using `List` (ArrayList), `Set` (HashSet), `Map` (HashMap); iterating, adding, removing and looking up elements.	Differences between implementations (ordering, duplicates, performance), generics, sorting with comparators, lambdas and method references, the Stream API (filter, map, collect), `Optional` and handling missing values.
Error handling	What exceptions are, try/catch/finally, common exceptions (NullPointerException, IllegalArgumentException), reading a stack trace.	Checked versus unchecked exceptions, custom exceptions, try-with-resources, when to throw versus handle, avoiding null through design.
Tooling and workflow	Using an IDE, compiling and running programs, basic debugging, Git basics (clone, commit, push, pull, branch).	Build tools (Maven or Gradle) and dependencies, Git workflows with branches and pull requests, writing and running unit tests with JUnit, understanding test results.
Web and REST APIs	What HTTP is, requests and responses, JSON, the idea of a REST API, the main HTTP methods (GET, POST, PUT, DELETE).	Designing simple REST endpoints, status codes, request validation, DTOs versus entities, documenting an API with Swagger or OpenAPI.
Spring Boot	Purpose of Spring Boot, project structure, running an application, simple controllers.	Dependency injection and beans, the controller, service and repository layers, configuration properties, using Spring libraries to call external services, mapping responses to Java objects.

Area	Level 1: foundation	Level 2: adds
Databases and data access	Relational database concepts: tables, rows, primary and foreign keys, basic SQL (SELECT, INSERT, UPDATE, DELETE, WHERE).	Joins and relationships, JPA entities and annotations, Spring Data repositories and query methods, limiting and sorting results, transactions at a basic level.
Code quality	Meaningful names, readable formatting, small methods, comments only where useful.	Single responsibility, avoiding duplication, simple refactoring, reading and reviewing someone else's code, reasoning about edge cases.

4. How the levels differ

- **Level 1** checks that the person knows the basic building blocks and can use them in short, single-concept tasks. Typical questions: name the right keyword, identify what a line of code does, pick the correct basic structure.
- **Level 2** checks that the person can combine several concepts, read slightly longer code, choose between options and explain why. Typical questions: predict the result of a code fragment, choose the best structure or design, spot a bug or a better approach.
- A question belongs to a level by the depth of reasoning it needs, not by the topic alone. The same topic can appear at both levels with different depth.

5. Out of scope

The following are not part of this competence unless a request explicitly includes them:

- JVM internals and tuning (garbage collector settings, bytecode, class loading).
- Advanced concurrency (thread pools, locks, reactive programming).
- Desktop and mobile UI frameworks (JavaFX, Swing, Android).
- Deployment and infrastructure (Docker, Kubernetes, CI/CD pipelines, cloud services).
- Frontend frameworks and languages other than Java.
- Language versions before Java 17, and preview features.

6. Guidance for writing questions

- Each question tests one clear idea and has one unambiguously correct answer.
- Use only topics listed above, at the depth of the requested level.
- Prefer short code examples and realistic situations over pure definitions.
- Wrong answer options should be plausible and based on common mistakes, not on trick wording.
- If a question needs knowledge from the out-of-scope list, it should not be generated.